

از پایه تا کاربردی RabbitMQ آموزش

.NET و پیاده‌سازی با Direct Exchange جزوه‌ی کامل مطالب یادگرفته‌شده در این گفت‌وگو — تمرکز بر

هنوز تمام نشده و بعد از این مرحله Direct Exchange **وضعیت جزوه**: این نسخه همه‌ی مطالبی را که تا این نقطه آموزش داده شده جمع‌بندی می‌کند. مبحث آموزش ادامه پیدا می‌کند.

RabbitMQ بخش ۱ — مفاهیم پایه

چیست؟ RabbitMQ ۱.

است؛ یعنی واسطه‌ای که پیام را از یک برنامه یا سرویس می‌گیرد و آن را برای مصرف سرویس دیگری مدیریت Message Broker ریت‌ام‌کیو یک می‌کند. این واسطه باعث می‌شود سرویس‌ها مجبور نباشند همیشه مستقیماً و هم‌زمان با یکدیگر ارتباط داشته باشند.

Payment Service ---> RabbitMQ ---> Notification Service

برای نمونه، سرویس پرداخت می‌تواند بعد از موفقیت تراکنش یک پیام منتشر کند و سرویس اعلان بعداً آن پیام را دریافت و پردازش کند.

چیست؟ Message ۲.

پیام همان داده‌ای است که بین سرویس‌ها جابه‌جا می‌شود. این داده می‌تواند اطلاعات یک پرداخت، شناسه کاربر، مبلغ، وضعیت تراکنش یا هر اطلاعات لازم باشد Consumer برای پردازش سمت.

```
{
  "paymentId": 123,
  "userId": 45,
  "amount": 250000,
  "status": "Paid"
}
```

چیست؟ Producer ۳.

را Producer نقش Payment می‌فرستد. در مثال این جزوه، سرویس RabbitMQ را ایجاد می‌کند و آن را به Message تولیدکننده سرویسی است که دارد.

چیست؟ Consumer ۴.

Notification دریافت می‌کند و بر اساس آن یک عملیات انجام می‌دهد. در مثال ما، سرویس RabbitMQ را از Message مصرف‌کننده سرویسی است که را دارد Consumer نقش.

چیست؟ Queue ۵.

باعث می‌شود Queue آن‌ها را دریافت و پردازش کند. وجود Consumer است که پیام‌ها در آن نگه داشته می‌شوند تا RabbitMQ صف محلی در لازم نباشد دقیقاً در یک لحظه در دسترس باشند Producer و Consumer.

Producer ---> [Queue: Message1 | Message2 | Message3] ---> Consumer

چیست؟ Exchange ۶.

Queue می‌فرستد، نه مستقیماً به Exchange معمولاً پیام را به Producer. مناسب است Queue به Message اکسچنج مسئول مسیریابی.

Producer ---> Exchange ---> Queue ---> Consumer

۷. Binding چیست؟

موردنظر Queue تحت چه شرایطی میتواند یک پیام را به Exchange است. این رابطه مشخص میکند Queue و Exchange بایدینگ رابطه‌ای بین هدایت کند.

۸. Routing Key چیست؟

می‌تواند از این مقدار برای تصمیم‌گیری درباره مقصد پیام Exchange ارسال می‌کند Message همراه Producer کلید مسیریابی یک مقدار متنی است که استفاده کند.

```
Routing Key = payment.success
```

۹. Direct Exchange چیست؟

فرستاده شود Queue به آن Message برابر باشد تا Queue یک Binding کلید مسیریابی پیام باید دقیقاً با کلید، Direct Exchange در

```
Routing Key = payment.success  
Binding Key = payment.success  
Result: Match -> Queue receives message
```

پیام را دریافت نمی‌کند Queue اگر مقدارها برابر نباشند، آن

۱۰. Fanout Exchange اشاره‌ی کوتاه به

شده ارسال می‌شود. این مفهوم فقط به‌صورت مقدماتی مطرح شد و Bind های Queue به تمام Routing Key پیام بدون توجه به Fanout Exchange در متمرکز است Direct Exchange ادامه آموزش فعلی روی

۲. NET. بخش — آماده‌سازی پروژه‌های

و دو برنامه Solution ساخت ۱۱

Consumer و یکی برای Producer در نظر گرفتیم: یکی برای Console Application برای تمرین، دو

```
dotnet new sln -n RabbitMqDirectDemo  
  
dotnet new console -n Payment.Producer  
dotnet new console -n Notification.Consumer  
  
dotnet sln add Payment.Producer  
dotnet sln add Notification.Consumer
```

ارتباط برقرار کنند Broker نصب می‌شود زیرا هر دو باید بتوانند با RabbitMQ برای هر دو پروژه، پکیج رسمی کلاینت

```
dotnet add package RabbitMQ.Client
```

۱۲. ConnectionFactory و Connection

```
var factory = new ConnectionFactory  
{  
    HostName = "localhost"
```

Direct Exchange در mandatory یادداشت‌های کامل تا مرحله — RabbitMQ آموزش

```
};  
  
using var connection = await factory.CreateConnectionAsync();
```

اتصال VirtualHost و Port، نام کاربری، رمز عبور، Broker تنظیمات اتصال را نگه می‌دارد؛ برای مثال آدرس ConnectionFactory کلاس ساخته می‌شود CreateConnectionAsync واقعی با

اتصال را باز نمی‌کند؛ فقط تنظیمات و سازوکار ساخت آن را آماده می‌کند ConnectionFactory نکته: خود

را طولانی‌عمر نگه Connection هزینه‌بر است و معمولاً TCP جدید ایجاد نمی‌کنیم، چون باز کردن اتصال Connection یک Message برای هر می‌داریم.

۱۳. Channel چیست؟

```
using var channel = await connection.CreateChannelAsync();
```

Exchange کردن Declare مثل RabbitMQ است و بیشتر عملیات Connection چنل یک ارتباط منطقی روی Queue، Publish و Consume TCP Connection باعث می‌شود برای هر عملیات یک Channel از طریق آن انجام می‌شود. استفاده از

Direct Exchange بخش ۳ — ساخت توپولوژی

۱۴. Declare کردن Exchange

```
await channel.ExchangeDeclareAsync(  
    exchange: "payment.direct",  
    type: ExchangeType.Direct  
);
```

با همین نام و مشخصات سازگار قبلاً وجود Exchange وجود دارد. اگر Direct و از نوع payment.direct ای به نام Exchange این کد مطمئن می‌شود داشته باشد، همان مورد استفاده می‌شود.

همان نمی‌تواند Exchange خطا می‌دهد. برای نمونه، یک RabbitMQ، همان‌نام از قبل وجود داشته باشد ولی مشخصات ناسازگار باشد Resource نکته: اگر شود Fanout Declare و بار دیگر Direct یکبار

۱۵. Declare کردن Queue

```
await channel.QueueDeclareAsync(  
    queue: "payment.notification.queue",  
    durable: true,  
    exclusive: false,  
    autoDelete: false,  
    arguments: null  
);
```

با همین نام و مشخصات سازگار موجود باشد، همان Queue می‌کند و اگر Declare را payment.notification.queue ای به نام Queue این کد استفاده می‌شود Queue

۱۶. durable پارامتر

است و Queue تعریف می‌شود. این تنظیم درباره خود RabbitMQ شدن Restart برای باقی ماندن بعد از Queue خود durable: true با مقدار ها را ندارد Message به‌تنهایی معنی ماندگار بودن

Direct Exchange در mandatory یادداشت‌های کامل تا مرحله RabbitMQ آموزش

```
durable: true
```

۱۷. پارامتر exclusive

است و با Connection محدود به همان Queue ایجادکننده وابسته می‌شود. در این حالت استفاده از Connection به Queue exclusive: true با مقدار استفاده کردیم exclusive: false اصلی سناریوی پرداخت از Queue حذف می‌شود. برای Queue، Connection بسته شدن

```
exclusive: false
```

۱۸. پارامتر autoDelete

را حذف Queue می‌تواند RabbitMQ، از آن جدا شود Consumer داشته باشد و آخرین Consumer قبلاً Queue وقتی autoDelete: true با مقدار صف حذف نشود Consumer شدن Down استفاده کردیم تا با autoDelete: false اصلی پرداخت از Queue کند. برای

```
autoDelete: false
```

۱۹. پارامتر arguments

می‌دهیم null استفاده می‌شود. وقتی به تنظیم اضافه‌ای نیاز نداریم، مقدار Queue برای تنظیمات اضافی arguments پارامتر

```
arguments: null
```

را تنظیم کرد Queue در Message می‌توان مدت مجاز ماندن x-message-ttl برای نمونه، با

```
var arguments = new Dictionary<string, object?>
{
    ["x-message-ttl"] = 60000
};
```

x- تنظیم دیگری مانند Queue برای انقضای Queue. است، نه انقضای خود Queue های داخل Message مربوط به انقضای x-message-ttl **نکته:** گزینه expires وجود دارد.

۲۰. ساخت Binding

```
await channel.QueueBindAsync(
    queue: "payment.notification.queue",
    exchange: "payment.direct",
    routingKey: "payment.success"
);
```

آن را به Direct Exchange، برسد Exchange به payment.success برابر Routing Key با Message می‌گیرد اگر Binding این مسیریابی کند payment.notification.queue.

Producer از Message بخش ۴ — ارسال

۲۱. به بایت Message تبدیل

```
var message = "Payment 123 succeeded";
```

```
var body = Encoding.UTF8.GetBytes(message);
```

تبدیل می‌شود. این byte به Publish قبل از JSON یا string را به صورت بایت دریافت می‌کند. بنابراین داده‌ای مثل Message بدنه RabbitMQ به نوع داده خاص یک زبان برنامه‌نویسی وابسته نباشد Broker موضوع باعث می‌شود.

۲۲. Publish کردن Message

```
await channel.BasicPublishAsync(
    exchange: "payment.direct",
    routingKey: "payment.success",
    body: body
);
```

های منطبق Binding برای پیدا کردن Direct Exchange کلیدی است که routingKey مقصد اولیه پیام را مشخص می‌کند. پارامتر exchange پارامتر به صورت بایت است Message محتوای واقعی body استفاده می‌کند و.

۲۳. Durable Queue در برابر Persistent Message

BasicProperties از Message است، ولی برای Queue برای خود durable: true دو موضوع جدا هستند. مقدار Message و Queue ماندگاری استفاده می‌کنیم.

```
var properties = new BasicProperties
{
    Persistent = true
};

await channel.BasicPublishAsync(
    exchange: "payment.direct",
    routingKey: "payment.success",
    basicProperties: properties,
    body: body
);
```

مفهوم / تنظیم	معنی
durable: true	باقی می‌ماند Restart Broker بعد از Queue خود
Persistent = true	می‌شود Publish به عنوان پیام ماندگار Message

را یاد Publisher Confirms بعداً، Publisher به تنهایی تضمین کامل تحویل نیست. برای اطمینان قوی‌تر سمت Message بودن Persistent: نکته می‌گیریم.

Consumer بخش ۵ — ساخت

۲۴. Consumer اتصال مستقل

خودش را می‌سازد Channel و Connection هم Consumer. مستقل خودشان را دارند Connection معمولاً Producer و Consumer

```
var factory = new ConnectionFactory
{
    HostName = "localhost"
};

using var connection = await factory.CreateConnectionAsync();
using var channel = await connection.CreateChannelAsync();
```

۲۵. Declare کردن Queue در Consumer

وجود دارد. مشخصات Queue بالا آمد، مطمئن شود Producer می‌کند تا اگر زودتر از Declare را با همان مشخصات Queue هم Consumer Declare موجود سازگار باشد Queue باید با Declare.

```
await channel.QueueDeclareAsync(
    queue: "payment.notification.queue",
    durable: true,
    exclusive: false,
    autoDelete: false,
    arguments: null
);
```

۲۶. ساخت AsyncEventingBasicConsumer

```
var consumer = new AsyncEventingBasicConsumer(channel);
```

دارد ReceivedAsync پیام‌ها استفاده می‌شود و رویدادی به نام asynchronous می‌سازد که برای دریافت Consumer Object این خط یک

۲۷. رویداد ReceivedAsync

```
consumer.ReceivedAsync += async (model, ea) =>
{
    Console.WriteLine("Message received");
    await Task.CompletedTask;
};
```

تحويل دهد Consumer را به Message یک RabbitMQ زمانی اجرا می‌شود که Handler این

۲۸. خواندن بدنه Message

```
var body = ea.Body.ToArray();
var message = Encoding.UTF8.GetString(body);
```

GetString فقط بدنه را در یک آرایه بایتی کپی می‌کند و ToArray تحويل داده شده است. متد Handler پیام قبلاً به ReceivedAsync در زمان اجرای تبدیل می‌کند string به UTF-8 بایت‌ها را با

۲۹. Queue روی Consumer ثبت

```
await channel.BasicConsumeAsync(
    queue: "payment.notification.queue",
    autoAck: true,
    consumer: consumer
);
```

را دریافت کند Queue موردنظر ثبت می‌کند تا بتواند پیام‌های آن Queue را روی Consumer، BasicConsumeAsync متد

۶ بخش — Ack، Nack خطا

۳۰. مفهوم autoAck

آن را تأییدشده در نظر می‌گیرد. اگر برنامه بعد از تحويل ولی قبل از پایان Consumer به Message به محض تحويل RabbitMQ، autoAck: true با کند، احتمال از دست رفتن پیام وجود دارد Crash پردازش

را به‌صورت دستی ارسال کند Ack، فقط بعد از پردازش موفق Consumer استفاده شود تا autoAck: false برای سناریوهای حساس بهتر است از

Direct Exchange mandatory یادداشت‌های کامل تا مرحله — RabbitMQ آموزش

```
await channel.BasicConsumeAsync(
    queue: "payment.notification.queue",
    autoAck: false,
    consumer: consumer
);
```

۳۱. BasicAckAsync

```
await channel.BasicAckAsync(
    deliveryTag: ea.DeliveryTag,
    multiple: false
);
```

با موفقیت پردازش شده است Message اعلام می‌کند که آن RabbitMQ به Message بعد از پردازش موفق Consumer، دستی Ack با DeliveryTag شود یعنی فقط همان یک multiple: false است و Channel روی Message شناسه تحویل همان DeliveryTag

۳۲. BasicNackAsync

```
await channel.BasicNackAsync(
    deliveryTag: ea.DeliveryTag,
    multiple: false,
    requeue: true
);
```

برمی‌گردد تا Queue دوباره به Message، باشد requeue: true موفق نبوده است. اگر Message اعلام می‌کند که پردازش Consumer، Nack با امکان پردازش مجدد وجود داشته باشد.

۳۳. تفاوت requeue: true و requeue: false

مفهوم / تنظیم	معنی
requeue: true	اصلی برمی‌گردد و می‌تواند دوباره تحویل داده شود Queue دوباره به Message.
requeue: false	تنظیم نشده باشد، Dead Letter اصلی بر نمی‌گردد. اگر Queue به Message معمولاً کنار گذاشته می‌شود.

ایجاد شود و Redelivery باشد، ممکن است یک حلقه بی‌پایان requeue: true به دلیل داده خراب همیشه شکست بخورد و هر بار Message اگر یک را بررسی می‌کنیم Dead Letter Queue محدود و Retry منابع سیستم مصرف شوند. برای چنین سناریوهایی بعداً

در چند مسیر Direct Exchange بخش ۷ — رفتار

۳۴. های متفاوت Routing Key با Queue چند

```
payment.success -> payment.notification.queue
payment.failed -> payment.failed.queue
```

دارد آن را دریافت می‌کند payment.failed برابر Binding Key ای که Queue منتشر شود، فقط payment.failed با Message اگر

۳۵. Routing Key یک Queue برای چند

```
Queue A -> payment.success
Queue B -> payment.success
Queue C -> payment.failed
```

را دریافت می‌کنند، چون هر دو Message هر دو Queue A و Queue B، پیام بفرستد payment.success برابر Routing Key با Producer اگر Binding منطبق دارند.

آن تطابق دقیق داشته باشد، یک نسخه Binding Key ای که Queue نمی‌فرستد؛ هر Queue را فقط به یک Message الزاماً Direct Exchange **نکته**: پس از دریافت می‌کند Message از

۳۶. پیام Unroutable

ای نمی‌شود. در حالت پیش‌فرض و بدون سازوکار دیگر، Queue وارد هیچ Message، پیام تطابق نداشته باشد Routing Key با Binding Key اگر هیچ. چنین پیامی می‌تواند از دست برود.

```
Existing bindings:
  payment.success
  payment.failed

Published routing key:
  payment.refund

Result:
  No matching queue
```

۳۷. گزینه mandatory

```
await channel.BasicPublishAsync(
    exchange: "payment.direct",
    routingKey: "payment.refund",
    mandatory: true,
    basicProperties: properties,
    body: body
);
```

برمی‌گرداند. در Publisher به basic.return آن را از مسیر RabbitMQ، نشود Route ای Queue به هیچ Message اگر mandatory: true، مربوط به پیام برگشتی را پیاده‌سازی کنیم Handler مرحله بعدی آموزش باید

تا نقطه فعلی Producer بخش ۸ — کد کامل

```
using RabbitMQ.Client;
using System.Text;

var factory = new ConnectionFactory
{
    HostName = "localhost"
};

using var connection = await factory.CreateConnectionAsync();
using var channel = await connection.CreateChannelAsync();

await channel.ExchangeDeclareAsync(
    exchange: "payment.direct",
    type: ExchangeType.Direct
);

await channel.QueueDeclareAsync(
    queue: "payment.notification.queue",
    durable: true,
    exclusive: false,
    autoDelete: false,
```

```

    arguments: null
);

await channel.QueueBindAsync(
    queue: "payment.notification.queue",
    exchange: "payment.direct",
    routingKey: "payment.success"
);

var message = "Payment 123 succeeded";
var body = Encoding.UTF8.GetBytes(message);

var properties = new BasicProperties
{
    Persistent = true
};

await channel.BasicPublishAsync(
    exchange: "payment.direct",
    routingKey: "payment.success",
    mandatory: true,
    basicProperties: properties,
    body: body
);

Console.WriteLine($"Sent: {message}");
Console.ReadLine();

```

هنوز در آموزش پیاده‌سازی نشده است `basic.return` مربوط به Handler فعال است، ولی `mandatory` نکته: در این کد

تا نقطه فعلی Consumer بخش ۹ — کد کامل

```

using RabbitMQ.Client;
using RabbitMQ.Client.Events;
using System.Text;

var factory = new ConnectionFactory
{
    HostName = "localhost"
};

using var connection = await factory.CreateConnectionAsync();
using var channel = await connection.CreateChannelAsync();

await channel.QueueDeclareAsync(
    queue: "payment.notification.queue",
    durable: true,
    exclusive: false,
    autoDelete: false,
    arguments: null
);

var consumer = new AsyncEventingBasicConsumer(channel);

consumer.ReceivedAsync += async (model, ea) =>
{
    try
    {
        var body = ea.Body.ToArray();
        var message = Encoding.UTF8.GetString(body);
    }
    catch { }
}

```

```

Console.WriteLine($"Received: {message}");

// Business operation

await channel.BasicAckAsync(
    deliveryTag: ea.DeliveryTag,
    multiple: false
);
}
catch
{
    await channel.BasicNackAsync(
        deliveryTag: ea.DeliveryTag,
        multiple: false,
        requeue: true
    );
}
};

await channel.BasicConsumeAsync(
    queue: "payment.notification.queue",
    autoAck: false,
    consumer: consumer
);

Console.WriteLine("Waiting for messages...");
Console.ReadLine();

```

Retry مناسب نیست. این کد صرفاً وضعیت فعلی آموزش را نشان می‌دهد و در ادامه باید Retry بدون محدودیت `requeue: true`، نکته: برای سیستم واقعی Retry مناسب نیست. این کد صرفاً وضعیت فعلی آموزش را نشان می‌دهد و در ادامه باید Retry بدون محدودیت `requeue: true`، نکته: برای سیستم واقعی و Dead Letter Queue را اضافه کنیم.

بخش ۱۰ — مرور سریع برای مصاحبه

مفهوم / تنظیم	معنی
RabbitMQ	برای انتقال و مدیریت پیام بین سرویس‌ها Message Broker یک.
Producer	می‌کند Publish را Message سرویسی که.
Consumer	را دریافت و پردازش می‌کند Message سرویسی که.
Queue	Consumer تا زمان دریافت توسط Message محل نگهداری.
Exchange	ها Queue پیام به Routing مسئول.
Binding	Queue و Exchange رابطه میان.
Routing Key	Direct Exchange می‌فرستد و در Message همراه Producer کلیدی که استفاده می‌شود Binding Key برای تطابق با.
Direct Exchange	Routing Key آن‌ها دقیقاً با Binding Key هایی می‌فرستد که Queue پیام را به برابر باشد.
Connection	RabbitMQ اتصال اصلی و نسبتاً پرهزینه شبکه‌ای به.
Channel	از طریق آن RabbitMQ که عملیات Connection ارتباط منطقی سبکتر روی انجام می‌شود.
durable	می‌شود Restart Broker بعد از Queue باعث ماندگاری خود.
Persistent	می‌کند Publish را به عنوان پیام ماندگار Message.

مفهوم / تنظیم	معنی
Ack	Message اعلام موفقیت پردازش
Nack	Message اعلام شکست پردازش
requeue	اصلی برگردد یا نه Queue شکست خورده به Message مشخص می‌کند
mandatory	برگردانده شود Publisher شدن به Route غیر قابل Message باعث می‌شود

باید یاد بگیریم Direct Exchange بخش ۱۱ — چیزهایی که در ادامه

- mandatory و BasicReturn مدیریت پیام برگشتی با
- Broker برای اطمینان بیشتر از پذیرش پیام توسط Publisher Confirms
- Consumer هر Unacked های Message برای کنترل تعداد QoS و Prefetch
- و رفتار توزیع پیام‌ها Queue روی یک Consumer چند
- Redelivery Loop محدود و جلوگیری از Retry
- Dead Letter Queue و Dead Letter Exchange
- Consumer کردن در Deserialize و JSON ارسال مدل‌های واقعی به صورت
- برای جلوگیری از اجرای دوباره عملیات کسب‌وکار Consumer در Idempotency
- با یک نمونه کاربردی کامل Direct Exchange جمع‌بندی نهایی